

Posse: A Cost-Aware Operating System for Long-Horizon, Human-Directed Multi-Agent Work

infra precinct • design_doc_and_repo_400
time-series-omp / OMPGen orchestration

2026-07-30 • design & experience report

Abstract

Most LLM-agent frameworks are built for a *single, supervised task*: a human starts a session, watches it run, and reads the result. The regime we actually operate in is different. Over weeks, one human asynchronously directs *dozens* of independent work threads by email; each thread must run unattended, survive process death and provider rate limits, remember what it has done, and — above all — not waste money. Naively porting a session-based framework to this regime is how a single project ran up a monthly bill that reached ~\$10k, much of it cold prompt-cache rewrites. This paper presents **Posse**, the “Sheriff & Deputies” system that runs our research group’s autonomous work. Posse targets four problems session-based frameworks do not address: (i) *continuity* of a single thread’s logic across interruptions and process restarts; (ii) *management* of many concurrent threads; (iii) *continuous, autonomous* operation with self-recovery; and (iv) *token efficiency* in long-lived threads. Its three design decisions are to *decouple a thread’s identity from process liveness*, *treat token cost as a first-class scheduling resource*, and *separate zero-cost mechanical control from paid cognition*. We describe each core module with the alternatives we considered and our rationale, marking honestly what is deployed versus designed-and-partly-realized, and report operational experience from a live deployment (six precincts, 400+ cases) — including the naming pathology that motivated the identity work (259 accumulated single-use worker identities) and the incident behind the cost model. Posse is not a new reasoning method but a systems answer to a question the field has largely left open: how do you keep a crowd of agents working, cheaply, for a long time, with a human in the loop but not on the clock?

1 Introduction

The last two years produced a rich toolkit for making a language model *act*: reasoning-and-acting loops [3, 4], autonomous task-list agents [1, 2], and multi-agent frameworks in which several role-specialized agents collaborate on one job [8, 10, 12, 13]. Almost all of this work optimizes the *inside* of a single task episode: how one agent, or one team of agents, reasons, uses tools, and coordinates from the moment a human presses “go” to the moment the task returns.

We found that running agents *in production for real research* stresses a different, mostly unaddressed layer — the *outside* of the task. In our group, a single human directs the work of many agents the way a professor directs students: by email, asynchronously, over weeks. A message arrives (“run the NOAA sweep,” “fix the dashboard bug,” “write this design doc”); an agent should pick it up, work it to completion (possibly over hours of GPU time), and email back a plan, milestones, and a result — all while the human is asleep, in a meeting, or directing ten other threads. When the human replies mid-task, the agent must incorporate the feedback without losing its place. When the machine reboots or the provider imposes a rate limit, the work must resume, not vanish. And none of it may quietly cost a fortune.

Concretely, four problems dominate this “outside” layer. They are the problems this document is organized around.

P1. Continuity of a thread. A single logical thread of work — one question and its follow-ups — must retain its context and its logic across interruptions, human feedback, crashes, and restarts, even when the underlying process has died and been replaced.

P2. Managing many threads. Dozens of threads run concurrently. Each needs a stable identity, an owner, isolation from the others, and a place in a structure a human can navigate.

P3. Continuous, autonomous operation. The system must run itself: route incoming work, keep workers alive, recover from crashes and rate limits, serialize contended resources (one GPU), and wake a worker when a long-running job finishes — without a human babysitting a terminal.

P4. Token efficiency in long threads. A thread that lives for days accumulates a large context. Re-reading or rewriting that context is the dominant cost, and the provider’s caching economics make the difference between cheap and ruinous a factor of $\sim 12.5\times$. Long-horizon operation is only viable if cost is engineered, not incurred.

This work. We present **Posse**¹, a deployed orchestration system that treats these four problems as its primary design targets. Its organizing metaphor is a *Sheriff* (a single autonomous system manager) supervising *Deputies* (ephemeral worker agents), grouped into *Precincts* (domains). Posse is deliberately *not* a new agent reasoning method; the deputies inside it are ordinary tool-using LLM agents. It is the layer around them — identity, memory, routing, recovery, and cost control — that is the contribution.

Contributions.

1. A *requirements analysis* for long-horizon, human-directed, multi-agent operation, and a comparison

¹The system was previously branded “Rookery”; “Posse” is the current name.

showing that popular frameworks — built for supervised single episodes — systematically miss it (§2).

2. Three design principles that resolve the tensions among P1–P4: *identity decoupled from liveness*, *cost as a scheduling resource*, and *zero-cost mechanical control separated from paid cognition* (§3).
3. A modular design realizing these principles — precinct memory (§4), deputy continuity (§5), the email control plane (§6), the Sheriff (§7), cost-aware scheduling (§8), multi-agent coordination (§9), and observability (§10) — the core modules presented with the alternatives we considered and why we chose as we did.
4. An *experience report* from a live deployment, including a cost incident (a monthly bill reaching ~\$10k), its root cause in prompt-cache economics, and the scheduling policy that fixed it (§8).

This is a design-and-experience report, in the tradition of operational-systems papers: we argue that the design meets the requirements and back it with deployment evidence, rather than with a head-to-head benchmark against frameworks that target a different regime.

2 Background and Limitations

2.1 The landscape of agent frameworks

Existing systems cluster into four families.

(a) Autonomous single-agent loops. AutoGPT and BabyAGI [1, 2] wrap one LLM in a plan-act-observe loop with tool use and a scratch/vector memory, aiming to pursue a goal unsupervised. Reasoning scaffolds — ReAct [3], Reflexion [4], Tree of Thoughts [5] — improve the loop’s decisions. Voyager [6] pushes toward *lifelong* operation with a growing skill library, but in a single embodied environment. These target the quality of one agent’s autonomous trajectory.

(b) Multi-agent collaboration. AutoGen [8], CAMEL [9], MetaGPT and ChatDev [10, 11], and CrewAI [12] run several role-specialized agents (planner, coder, reviewer; or a simulated “software company”) that converse or follow a standard operating procedure to complete one project. The contribution is division of labor within an episode.

(c) Graph / workflow orchestration. LangGraph [13] and similar libraries let a developer wire agents and tools into a stateful graph with checkpointing and human-in-the-loop interrupts. OpenAI’s Swarm [14] offers minimal stateless routines with explicit hand-offs. These are *construction kits*: they execute a graph when invoked.

(d) Agent operating systems and memory managers. A recent line argues the LLM should be treated like a process under an OS. AIOS [16] proposes a kernel that schedules agent “processes” and manages context, memory, and tools. MemGPT [17] treats context like virtual memory, paging information between the prompt and external stores. Generative Agents [7] contribute a memory stream with reflection and retrieval. These are

the closest in spirit to Posse and we return to them in §11.

2.2 Requirements

From operating agents for real research we distilled seven requirements. They follow directly from P1–P4 plus the operational reality of one human, one machine, a metered API, and a work stream that never quite ends.

R1 Long-horizon continuous operation. Run unattended for days to weeks, not for one session.

R2 Durable thread identity & continuity. A thread keeps one identity and its context *across process death* (P1).

R3 Concurrent thread management. Many independent threads, each isolated, owned, and navigable (P2).

R4 Asynchronous human-in-the-loop. A human directs and *interrupts* threads asynchronously, without holding a session open (P1, P3).

R5 Autonomous fault recovery. Self-heal from crashes and provider rate limits (P3).

R6 Cost-aware scheduling. Treat token/\$ cost as a scheduled resource, not an afterthought (P4).

R7 Bounded, auditable memory. Long-term memory that stays within the model window and is trustworthy enough to act on (P2, P4).

These are not fully orthogonal: R2, R3, and R7 are three facets of one *durable-record substrate* (identity, ownership, and per-thread memory), and R1 is unreachable without R5. We list them separately because prior systems fail them *independently* — a system can manage many threads (R3) yet lose each one’s continuity on restart (R2). That Posse answers R2/R3/R7 with a *single* durable-record mechanism is a feature of its design, not an artifact of the taxonomy.

2.3 Why existing approaches fall short

The families above are excellent at what they target and largely silent on R1–R7. The mismatch is structural, not incidental: they assume a human is *present* for the duration of a *single* episode.

- **Identity is tied to a live process.** In every framework we know, a “thread” is a running object (a Python process, a graph run, a chat). When it ends — normally or by crash — its identity and live context end with it (fails R2). Continuity across restarts is left to the application.
- **No notion of asynchronous interruption.** Human-in-the-loop in LangGraph or AutoGen means *the run pauses and waits* for input. None models a human who is absent for hours and then injects feedback into a thread that has meanwhile been killed and restarted (fails R4).

- **No self-recovery over long horizons.** Frameworks retry a tool call; none, as a system, detect that a worker died at 3 a.m. or hit a weekly rate limit and relaunch it after the reset (fails R1, R5).
- **Cost is unmanaged.** Cost appears, if at all, as a per-call token count. No framework we surveyed schedules, as an orchestration-layer policy, around the *provider’s caching economics*, where the gap between two touches of a context decides whether you pay $1\times$ or $\sim 12.5\times$ (fails R6). This is precisely where our \$10k incident came from (§8).
- **Memory is either unbounded or per-episode.** Standing-agent designs grow one context without bound until it exceeds the window; per-episode designs forget everything at the end. AIOS/MemGPT address the *mechanism* of bounded memory but not the *operational* question of a separate, zero-cost manager that compacts durable per-domain records (partially meets R7).

Table 1 maps where each design puts its effort. Two rows deserve comment. *Durable-execution engines* (Temporal, AWS Step Functions) are the closest true prior art for R1–R3/R5 — durable identity and resume-across-crash are exactly their business — but they schedule *compute* durability, with no notion of LLM *token* cost (R6) or bounded LLM *memory* (R7), and the absent, asynchronous human (R4) is not their model. *OpenAI Assistants* give durable server-side thread identity (R2) but leave management, recovery, cost, and memory to the application. The gap Posse targets is the *conjunction* of R4, R6, and R7 with the durable-identity substrate — reusing the reasoning and multi-agent ideas of families (a)–(c) *inside* its deputies, the durable-thread idea of workflow engines, and the OS/memory stance that families (d) point toward. Posse’s own $\circ$ on R2 is deliberate: the durable-identity substrate ships, but the automatic continuity switch does not yet (§5).

3 Design Overview

3.1 Three insights

The requirements pull against each other. Full continuity (R2) wants one long-lived context; cost (R6) and the finite model window punish exactly that. Autonomy (R1, R5) wants always-on processes; the same always-on-ness is what a naive design bills you for. Posse rests on three decisions that resolve these tensions.

I1. Decouple identity from liveness. A thread’s identity is a *durable record*, not a running process. Continuity (R2) is provided by persisted case files and a lineage map, so a thread can be resumed — or rebuilt into a fresh process under the same identity — long after its original process died. This is the single idea that unties P1 from P3.

I2. Make token cost a first-class scheduling resource. Every waiting decision (poll vs. sleep), every memory decision (keep vs. compact), and every model choice (cheap vs. capable) is taken against an explicit cost model derived from the provider’s cache economics (R6). Cost is scheduled, not discovered on the invoice.

I3. Separate zero-cost mechanical control from paid cognition. The control plane — routing, liveness, recovery, waking, resource serialization, compaction *triggers* — makes *no* LLM calls. The LLM is invoked only for actual work and for a handful of genuinely judgment-bearing manager decisions. This is what makes always-on operation (R1) affordable (R6).

3.2 System overview

Figure 1 shows the result. Work is organized into **precincts** (domains: **eval**, **omp**, **query**, **paper**, **infra**, and a **receptionist** front desk). Each precinct owns durable *records*: a compacted *ledger* (big-picture digest), an append-only *case log* (index of finished work), and per-thread *case files* (§4). We distinguish a *thread* — the durable identity and lineage of a line of work — from a *case* — one numbered unit of work on it; a thread is a sequence of cases. Each case is worked by a **deputy**: an ephemeral LLM worker that lives for the duration of one case, emails the human a plan/milestones/final, and closes the case by appending to the records (§5).

The human’s channel is **email** (Fig. 1, top). An always-on *inbox router* maps each message — mechanically, with no LLM — to an existing thread (reply), a new deputy in a tagged precinct, or the receptionist (§6). Overseeing everything is the **Sheriff** (Fig. 1, left): a single *system manager* that supervises deputy liveness and precinct-records health. The Sheriff’s monitoring is zero-API (I3); it spends tokens only to compact a ledger or to decide a deputy’s change-request. Underneath sit the always-on daemons that provide routing, relaunch, job event-wakes, and GPU serialization — three (inbox, watchdog, job manager) supervised as the core control plane, the Sheriff and GPU manager self-restarting via idempotent scripts; a read-only **dashboard** (right, a separate process) observes it all.

3.3 Life of a case

Figure 2 traces one case end to end. An email arrives and is routed; a deputy is spawned or an existing identity resumed. The deputy emails a plan, works in small chunks, offloads compute (>2 min) to a detached job, and — for a long job (>50 min) — *submits it to the job manager and sleeps*, to be woken by an event rather than by polling (§8). It emails milestones. If the human replies mid-case, the deputy is *surgically interrupted*: its process is killed and relaunched with the new message injected, while its detached jobs keep running (§6); on resume it reads its mailbox *first*. Finally it emails a result and *closes* the case: one ledger paragraph, one case-log line, and a done-sentinel. Throughout, continuity (the dashed band) is carried not by the process — which may have

Table 1. Design emphasis in the operational regime of §2.2 (long-horizon, human-directed, multi-thread) — *not* a quality ranking; every system excels in the regime it targets. Rubric: ✓ = a first-class system property; ◦ = partial, or present but left to the application; × = not a concern of the design. Cells reflect published/observed features; borderline cases are marked ◦. We give ourselves an honest ◦ where a mechanism is designed but only partly deployed (§5).

	Type	R1 horizon	R2 identity	R3 threads	R4 async H-in-loop	R5 recovery	R6 cost	R7 memory
AutoGPT / BabyAGI [1, 2]	auton. loop	◦	×	×	×	×	×	◦
ReAct / Reflexion [3, 4]	reasoning loop	×	×	×	×	×	×	◦
Voyager [6]	lifelong single	✓	◦	×	×	◦	×	◦
Generative Agents [7]	memory / sim	✓	◦	◦	×	×	×	✓
AutoGen [8]	multi-agent chat	×	×	◦	◦	×	×	×
CrewAI / MetaGPT [12, 10]	role crew / SOP	×	×	◦	×	×	×	×
LangGraph [13]	graph orchestration	◦	◦	◦	◦	◦	×	◦
OpenAI Swarm [14]	hand-off routines	×	×	◦	×	×	×	×
OpenAI Assistants [18]	server threads	◦	✓	◦	◦	×	×	◦
Claude Code subagents [15]	in-session spawn	×	×	✓	◦	×	◦	×
Hosted SWE agents (Devin-style)	hosted autonomous	◦	◦	◦	◦	◦	×	◦
Durable exec. (Temporal/ Step) [19, 20]	workflow engine	✓	✓	✓	◦	✓	×	×
AIOS [16]	agent OS (kernel)	◦	◦	✓	×	◦	◦	✓
MemGPT [17]	memory OS	◦	◦	×	×	×	◦	✓
Posse (this work)	agent ops	✓	◦	✓	✓	✓	✓	✓

died and been replaced — but by the durable case file and a resume-or-reinject decision (§5). Underneath, the Sheriff and daemons keep the deputy alive and wake it, all without an LLM call.

The remaining sections detail each module: what it does, the design alternatives, and why we chose as we did.

4 Precincts and Bounded Memory

Role (R7, R2, P2/P4). A precinct is a domain of work with durable, structured memory. Its three record types form a memory hierarchy from most- to least-compacted:

- **Ledger** — a bounded, big-picture digest of the precinct (the “standing knowledge”), rewritten by compaction as it grows.
- **Case log** — an append-only index, one line per finished case. Never rewritten (except by the Sheriff’s surgical retraction), so attribution stays honest.
- **Case files** — the full per-thread record, the least-compacted and the source of truth a resumed thread reads.

All access goes through a single concurrency-safe *records manager* (`scratch_records.py`), never raw files. This is the deliberate choke point that makes the append-only contract enforceable: a deputy may only *append* (one ledger paragraph and one case-log line, at close); only the Sheriff may compact a ledger, retract a log line, or create/delete a precinct.

Why a manager, not a database? The records are human-readable Markdown/TSV so a human (or a resumed deputy) can read a precinct’s history directly, and so the dashboard can render it without a query layer. The manager gives us atomicity and role enforcement (flock + atomic replace) without the operational weight of a DB — appropriate for a single-machine deployment.

Design alternatives for long-term memory. The crux is R7 vs. the model window and R6:

1. *One growing context per domain* (the “standing agent” design). Maximal continuity, but the context grows without bound, eventually exceeds the window, and — per I2 — becomes the most expensive thing to touch. This is exactly why our long-lived **paper** thread had to be pinned to a smaller/cheaper model.
2. *Per-episode memory only*. Cheap and bounded, but forgets everything between cases; every follow-up must be re-briefed by hand (fails R2, R7).
3. *Retrieval over a vector store*. Scales, but retrieval is lossy and opaque, and it is hard for a human to audit “what does this domain know?”
4. **Chosen: a compacted ledger + append-only log + full case files**, managed by a separate zero-cost process. The ledger stays within a token budget (bounded, cheap to load); nothing is silently lost because the case log and case files are never compacted away; and the whole thing is plain text a human can read. Compaction (an LLM “reflection” over the domain, §7) is a coarse, operations-level analog of MemGPT’s paging [17] and Generative Agents’ reflection [7], but performed by the manager, not the worker, and at



POSSE

autonomous multi-agent operations

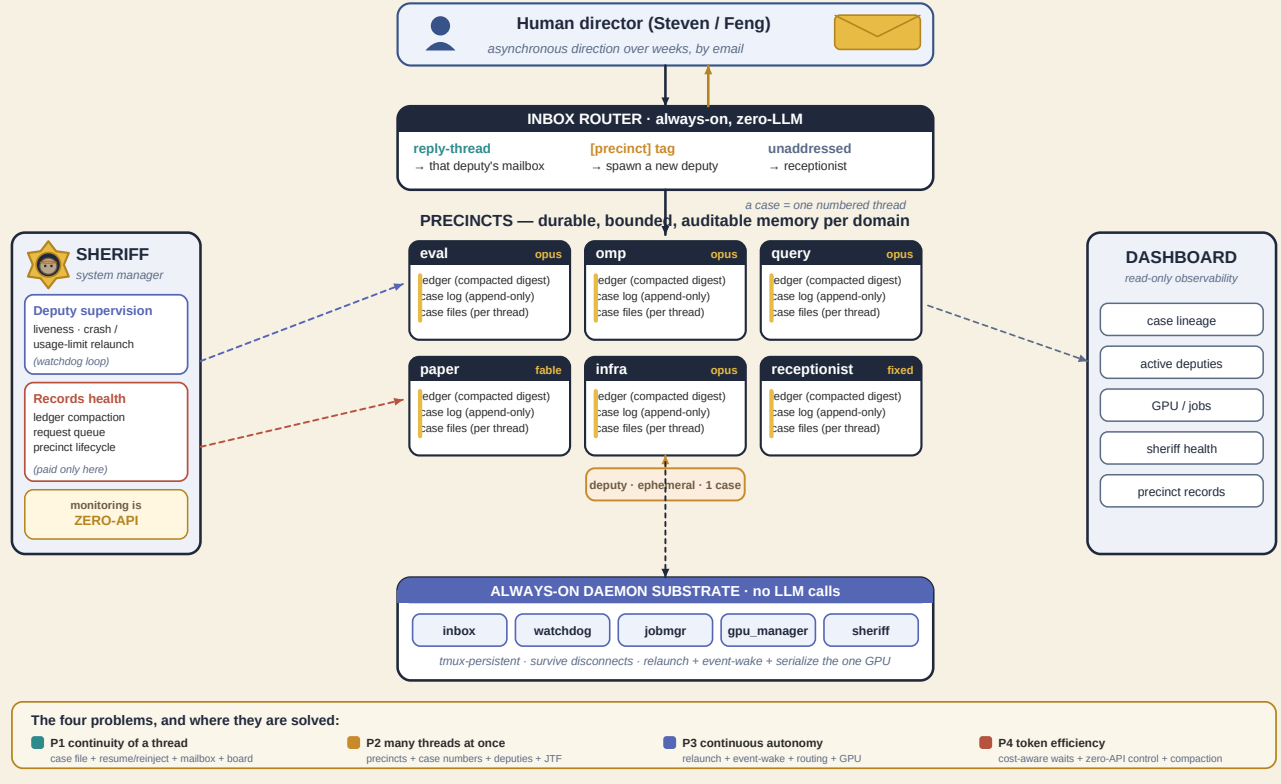


Figure 1. The Posse architecture. Work flows down the center (human → email → router → precincts → deputies), with the always-on daemon substrate underneath it. The *Sheriff* (left) manages the system with zero-API monitoring, spending tokens only on ledger compaction and change-request decisions; the *dashboard* (right) observes read-only. The bottom band maps the four problems (P1–P4) to the modules that solve them.

domain granularity.

Each precinct also carries a *default model* (e.g. the capable **opus** for most, the cheaper-but-capable **fabl**e for the token-heavy **paper** precinct), so cost/capability tiering is a per-domain policy (§8).

5 Deputies and Thread Continuity

Role (P1, R2). A deputy is an ordinary tool-using LLM agent that owns *one case*. The hard part is not what it does but how its *thread* survives everything that happens to the process running it.

The failure we started from. Early on, a thread’s identity *was* its process. A worker that finished touched a done-sentinel, which ended its session; the next follow-up therefore could not find it alive and spawned a brand-new worker with a fresh name and blank context, re-briefed by a hand-written spec. The result was 259 mostly single-use identities² and a project like the dashboard scattered across seven differently-named agents — continuity tied to *process liveness* rather than to the work. Exactly

²The current registry count; the pathology grew from ~235 at the time of an earlier internal analysis. Both are high-water marks of the same accumulation.

one thread (**paper**) escaped this, via a hard-coded fixed-session id, and alone spanned dozens of cases under one identity — the historical proof that stable identity was achievable. (That special case has since been *retired*: **paper** is now an ordinary precinct-routed deputy, so the general mechanism below — not a hard-coded exception — is what must carry continuity.)

The principle (I1). Route every follow-up to a *stable, meaningful identity* recorded in a persisted map — not to whatever process happens to be alive. Continue that identity by *context budget*: while its context is cheap, **-resume** the same session (full live context, warm cache); once it grows past budget, spawn a fresh process under the *same* identity and inject a compact brief (prior case numbers, keywords, pointers to the prior prompt, emails, and deliverables). Identity is stable either way; context stays bounded; the expensive cold revive of a giant context is avoided.

Design alternatives (continuity policy). We considered five points on the continuity–cost spectrum:

- Status quo* (reuse-if-alive, else new agent). Simplest, naturally parallel, bounded per-agent context — but continuity is accidental and naming is chaotic.
- Standing domain agents* (never die). Strong conti-



Life of a case

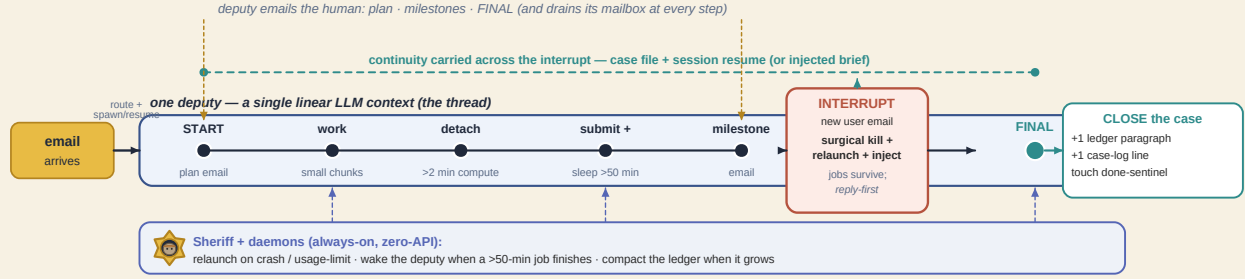


Figure 2. Life of a case. A deputy is one linear LLM context. Continuity survives the interrupt (top dashed band) via the durable case file plus session resume or an injected brief; the always-on Sheriff/daemon layer (bottom) relaunches, event-wakes, and compacts — with no LLM calls.

nuity and one obvious identity per domain, but one unbounded context: window pressure, cross-task interference, and serialization.

- C. *Per-lineage persistent agent* (resume the exact session on demand). Precise continuity and a stable per-thread identity, but inherits B’s unbounded growth on long threads.
- D. *Fresh agent per task + injected context*. Always bounded and cheap, lineage explicit, but the injected brief is a summary, not the live session, and naming multiplies unless paired with a stable name.
- E. **Chosen direction: hybrid.** Stable per-thread *name* (like C/D), and a *resume-if-cheap-else-reinject* switch (C when cheap, D when not), with the map persisted at creation. Among the five, only E avoids a *severe* weakness on the first three axes (continuity, identity, cost) by our qualitative assessment, and it subsumes **paper**’s retired special case as one instance of a general mechanism rather than a hard-coded exception.

The trade-off E accepts is a policy knob (the context-size threshold) and slightly more moving parts, all additive. We judged that worthwhile: it makes the continuity/cost trade-off *explicit* instead of an accident of which process is alive.

Deployment status (honest accounting). The *durable* half of E is shipped: a persisted **task** → **parent** → **precinct** lineage map records each thread’s identity and precinct at creation, and the active-deputies board (next) makes the current owner authoritative. The *context-budget switch* — metering a thread’s context size to choose resume-vs-reinject automatically — is designed but only *partly realized*: today the handler resumes a thread’s recorded session when it is still valid and otherwise spawns a fresh process under a routed identity, without yet measuring context size to drive the choice. So E is our chosen direction and its identity substrate is live; the automatic cost-threshold is future work. We flag this rather than imply more maturity than exists.

The board. A deputy’s current case is published to an *active-deputies board* (`active_deputies.json`) that the dashboard trusts over any stale launch-time default — so “who is working what, right now” is a single authoritative read (R3), independent of process state.

6 The Email Control Plane

Role (R4, P1/P3). Email is Posse’s human interface and, crucially, its *interrupt* mechanism. We chose it deliberately: it is asynchronous by nature (the human is never blocked and never blocking), it is durable (the mailbox is the queue), it threads (a reply carries its lineage), and it needs no bespoke UI.

Routing. The routing *decision* is mechanical — zero-LLM (I3) — on two signals: (1) a reply threaded to a deputy’s earlier email → that deputy’s *mailbox*; (2) an explicit [**precinct**] tag → a new deputy in that precinct (a reply to an already-*dead* deputy is attributed to its precinct via the persisted parent → precinct map — a third mechanical lookup). Only genuinely *unaddressed* mail falls to the receptionist, whose disposition (answer inline, or write a spec and spawn a deputy) is the one

routing path that spends an LLM turn. An earlier design instead scored message text against all workers with a keyword matcher; we removed it after it hijacked, for example, query mail to a long-dead agent. Deterministic routing is cheaper and more predictable, at the cost of an occasional receptionist hand-off (§11, L4).

Interruption. When a reply targets a *live* deputy, the message is appended to its mailbox under a lock and the deputy is *surgically interrupted*: only its LLM process is killed and immediately relaunched with the mailbox injected; its detached CPU/GPU jobs — separate processes — keep running and are reconciled on resume. On *any* resume the deputy’s first action is to drain its mailbox and reply (the “reply-first” rule), so human feedback is never processed on a stale plan. The delivery guarantee is *at-least-once with an idempotent drain*: the append and a boot marker are flock-serialized, so a burst of replies is appended in order and drained *together* on the next boot rather than lost or double-processed, and a job that finishes *during* the interrupt window is reconciled from its result file on resume, never re-submitted.

Design alternatives.

- *Kill everything vs. surgical kill.* Killing the whole session would kill in-flight GPU jobs (hours of compute). We kill only the claude process and let jobs survive as orphans with their output still streaming — the single most important robustness property of the interrupt path.
- *Debounce vs. immediate.* We removed an anti-churn cooldown: every human email interrupts *immediately*, with boot-marker coalescing so a burst of replies is drained together rather than lost. Latency of feedback matters more than avoiding a few extra relaunches.
- *Prompt via argv vs. stdin.* A subtle self-kill bug (`pkill` matching an agent’s own resume prompt embedded in its `argv`) taught us to pass prompts via *stdin*, never the command line — a reminder that at this scale the control plane’s own mechanics are a source of faults.

7 The Sheriff: Autonomous System Management

Role (R1, R5, R7). The Sheriff is the system’s single “manager” *role*, presented as one but implemented today as *two* cooperating always-on loops: a battle-tested liveness *watchdog* (deputy supervision — crash and rate-limit recovery) and a records daemon (precinct-records health — ledger compaction, the change-request queue, and precinct lifecycle). We deliberately unified the *model* (one manager, one dashboard panel) while keeping the two proven loops physically separate; folding them into a single process is a deferred option (see *Design alternatives* below).

The hard guarantee (I3). The Sheriff’s *monitoring is zero-API*. It scans sentinels, tmux liveness, and ledger token-lengths mechanically; it makes an LLM call *only* to (a) compact a ledger that crossed its soft limit, or (b) approve/deny a deputy’s change-request. An idle pass costs nothing. This is what lets the manager run forever without being a cost center.

Compaction. When a ledger crosses a soft token budget ($A = 20,000$) the Sheriff compacts it toward a target ($B = 10,000$) with one LLM call on a cheap global model (**fable**), given the whole-system context (all precincts’ ledgers) and a fixed system prompt that forbids inventing or silently dropping a case. On any failure or rate limit it falls back to a mechanical truncation, so the ledger is *always* bounded and the daemon never crashes. Compaction is the system’s “reflection” step — but performed by the manager, off the worker’s critical path.

Change-requests & authority. Records mutations that are not simple appends (retract a log line, edit a ledger, allocate a case number, create/delete a precinct) are *sheriff-only*. A deputy requests them via a queue; the Sheriff verifies the requester’s identity by matching its (`deputy`, `session`) against the live registry — the session UUID minted at spawn *is* the shared secret, so a forged deputy or a wrong session is rejected *before* any LLM call — then decides with one LLM call returning strict JSON, performs the op, and journals it. A failed or rate-limited decision *defers*: it never auto-approves and never deletes by default. Destructive operations add real-world gates: deleting a precinct requires no active deputies, an emailed “reply YES” confirmation with a timeout (matched *mechanically* by the zero-API daemon, not routed to an LLM), and a 14-day recoverable trash before hard purge.

Design alternatives.

- *Merge the watchdog into the Sheriff, or keep them separate?* We chose to *present* them as one manager but keep the battle-tested liveness loop physically separate (a staged “rebrand-first” path), because the relaunch logic is the most safety-critical code in the system — it is what prevents repeats of the \$10k incident and a past 42-hour dead-worker gap. Unifying the model without rewriting the mechanism captured the clarity at near-zero risk.
- *Mechanical vs. LLM compaction.* Mechanical truncation is free and deterministic but loses the “big picture”; an LLM rewrite preserves it but can fail. We use LLM-by-default with a mechanical fallback — quality when possible, safety always.
- *Who may edit records?* Letting deputies edit their own history would destroy attribution. Concentrating all non-append authority in one identity, gated by verified requests and a journal, keeps the record trustworthy.

8 Cost-Aware Scheduling

Role (R6, P4). This is where Posse most departs from prior frameworks: *token cost is scheduled*. The policy is

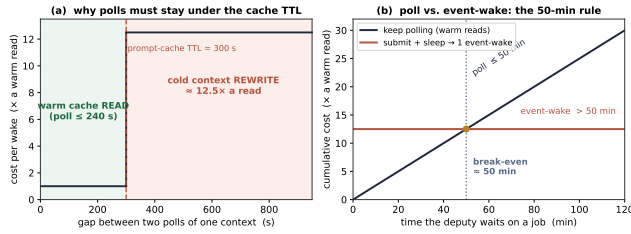


Figure 3. The cost model behind Posse’s wait policy (a *schematic* of the economics, not a trace of measured traffic). (a) A poll within the 300s cache TTL is a warm read; past it, a cold rewrite at $\sim 12.5\times$. (b) Cumulative cost of waiting: repeated warm polls grow with duration, one event-wake is a single cold rewrite; they cross at ~ 50 min — the threshold between the two strategies.

grounded in an incident.

The \$10,000 incident. Early in the deployment, workers waiting on long jobs polled their result files on a ~ 10 -minute cadence, sleeping between checks and re-reading their (large) context each time. The monthly bill reached $\sim \$10k$, roughly $\$6k$ of it traced to these cold rewrites. The root cause was not the number of polls but their *spacing* relative to the provider’s prompt cache [25]. The cache has a **300-second TTL**: a touch within 300s of the last is a warm *read*; a touch after is a cold *rewrite* of the whole context, at roughly $12.5\times$ the read (for our backend and context sizes; Fig. 3a). A 10-minute cadence guaranteed every single poll was a cold rewrite of a growing context.

The policy. From this model, two rules fall out:

1. **Short waits (≤ 50 min): poll, but under the TTL.** Sleep in chunks of *at most* 240s so every wake is a warm read. The old 10-minute chunks were the entire problem.
2. **Long waits (> 50 min): submit and sleep.** Hand the wait to a zero-API job manager and *end the turn*; the manager wakes the deputy by *event* when the job finishes (one cold rewrite, once), rather than paying a warm read every few minutes for an hour.

The 50-minute threshold is the break-even (Fig. 3b): at the 4-minute cap, warm polling costs *at most* ~ 15 reads/hour of context; an event-wake costs one rewrite (≈ 12.5 reads). The break-even is therefore $t^* \approx m/r$ (rewrite-multiplier over polls-per-hour) = $12.5/15\text{h} \approx 50$ min; below it, polling is cheaper; above it, event-wake wins. Crucially, the *context size cancels* — both a read and a rewrite scale with it — so t^* is independent of how large the thread has grown, which is what makes a single threshold correct for every deputy.

Two more cost levers.

- **Zero-API control plane (I3).** Every always-on daemon — router, liveness, job manager, GPU serializer, and the Sheriff’s monitoring — runs without an LLM call. Only real work and rare manager decisions are billed.
- **Model tiering.** Cheap-but-capable models (**fab**le) run

the token-heavy long threads (the **paper** precinct) and every Sheriff call; capable models (**opus**) run the rest. Tiering is a per-precinct default, overridable per case.

Why this generalizes. The constants (300s, $12.5\times$, 50 min) are provider-specific, but the *principle* generalizes to any backend with a *TTL-bounded* read/rewrite discount: there is a break-even m/r between “keep the context warm” and “tear down and event-wake.” (Backends with a persistent KV cache or a per-write surcharge change or remove it, and need the model re-derived.) Provider- and serving-layer prompt caching is of course well known [25]; what we have not seen elsewhere is an *agent-orchestration* layer that lifts this break-even into the scheduler’s *wait/park* decision.

9 Multi-Agent Coordination

Role (P2, R3). Most cases are one deputy, but three coordination primitives handle work that is not.

Sub-deputies. A deputy may spawn a *tracked* sub-deputy for a sub-task (numbered as a child of the parent case, e.g. 400a). Sub-deputies file their own records and appear on the board, so a large effort decomposes into a visible tree rather than an opaque recursion.

Sheriff-wake on completion. A parent that must wait for sub-deputies does *not* busy-wait (which would burn context per I2). It registers “wake me when X finishes” and parks; the Sheriff detects the sub-deputies’ done sentinels and wakes the parent with a “subtasks done” message. This is the event-wake idea (§8) applied to inter-agent dependencies.

Joint Task Forces (JTF). For work needing several agents at once, a JTF names a *lead* plus collaborators (and an optional critic). Each slot is either a *precinct* (spawn a fresh deputy there) or a *specific deputy* (a “must-take” assignment). The interesting case is assigning work to a deputy that is already busy on another case: interleaving two live cases in one linear context is unsafe. Our rule (chosen over “refuse if busy” and “context-switch now”) is *stop, hand over, then take*: the busy deputy hands its current case to a new deputy with full context injected (reusing the continuity machinery of §5), notifies the Sheriff, and only then takes the JTF role as a fresh case. The must-take assignment rides the *same* mailbox-and-interrupt machinery as an email reply, so it is never lost or duplicated even under a simultaneous human interrupt.

Anonymous helpers. For a throwaway helper whose result only the spawning deputy consumes — most often a *critic* that reviews a draft — a deputy may spawn an unregistered *anonymous* worker it owns and monitors itself. It files no paperwork and is invisible to the Sheriff, keeping the tracked record clean while still enabling the adversarial-critic pattern (a deputy reviewing its own draft with independent, disposable critics).

10 Observability

Role (R3, P2). A read-only web dashboard renders the system for the human: the case-lineage forest (which case followed which), the active-deputies board, GPU/job

state, Sheriff health, and each precinct’s records. It is deliberately *read-only over the same records the system runs on* — it computes nothing the system does not already persist — so it can never disagree with reality or perturb it. Where earlier versions *reconstructed* lineage heuristically after the fact, the persisted lineage map (§5) now makes that view authoritative. The dashboard is how one human keeps dozens of threads legible.

11 Discussion, Related Work, and Limitations

Positioning. Posse borrows freely: its deputies are ReAct-style tool-agents [3]; its JTFs echo role-based multi-agent work [8, 10, 12]; its ledger compaction is a coarse cousin of MemGPT’s paging [17] and Generative Agents’ reflection [7]; and its “LLM-as-managed-process” stance is the one AIOS [16] argues for. Its *systems* lineage is older still: the Sheriff’s supervise-and-relaunch loop is a supervision tree in the Erlang/OTP “let it crash” tradition [22, 23]; the read-only dashboard over the same records the system runs on is a reconciliation / desired-state pattern in the spirit of cluster controllers [24]; and durable identity that survives process death is exactly what durable-execution engines (Temporal, Step Functions, Airflow) [19, 20, 21] give ordinary workflows, while server-side threads [18] give it to conversations. What is new is the *operational synthesis* for long-horizon, human-directed *LLM* work: those engines schedule compute durability, not *token* cost, and none of them models a human who is *absent by default*, reachable only through an *email* control-and-interrupt plane. The one piece we have not seen elsewhere is scheduling an agent’s wait/park decisions against the provider’s *cache-TTL* economics (§8) — provider caching itself is well known [25], but not, to our knowledge, as an orchestrator policy. Where AIOS proposes a kernel and MemGPT a memory manager, Posse is a deployed system whose scheduler’s scarce resource is dollars, not GPU cycles.

Failure modes and open risks. Distinct from the scoping limitations below, four risks are inherent to the design. (i) *The Sheriff is a single point of failure* — *quis custodiet ipsos custodes?* It is a root daemon restarted only by an idempotent supervisor script (as the other daemons are), not itself watched by a peer; a silently-dead Sheriff halts compaction and recovery until a human notices. (ii) *Unsupervised work quality.* Posse keeps deputies alive and cheap; it does not judge whether their output is *correct*. A deputy can do the wrong thing for hours before the human reads its email — quality gating is orthogonal and out of scope here. (iii) *Interrupt-race correctness.* The at-least-once idempotent drain (§6) is the invariant that makes concurrent human interrupts safe; a bug there (we hit one — an agent that `pkilled` itself by matching its own resume prompt in `argv`, fixed by passing prompts via `stdin`) reintroduces lost or duplicated feedback, so this path needs the care of any lock protocol. (iv) *Compaction fidelity.* LLM ledger compaction can

drop or distort a case despite a prompt forbidding it; the mechanical-truncation fallback bounds *availability* but not *fidelity*, with the never-compacted case log as the backstop of record. We observed one compaction fault (a cache false-positive that misclassified a success as a rate limit), contained by the fallback.

Limitations. (1) Posse is a single-machine, single-human, single-provider system; the cost constants are provider-specific and multi-tenant fairness is out of scope. (2) We report operational *experience*, not a controlled benchmark against other frameworks — a fair comparison is hard precisely because they target a different regime, so our claims are about *design fit*, not measured speedups. (3) The continuity threshold and several policies are hand-tuned; learning them is future work. (4) Mechanical routing (I3) is less flexible than an LLM router for genuinely ambiguous mail — we accept occasional receptionist hand-offs as the price of predictability. (5) *Artifact.* The system runs on private research infrastructure; the design, scripts, and this document are internal, so we offer this as an experience report rather than a released, reproducible artifact.

Experience. The design has run our group’s autonomous work across six precincts and 400+ cases; the cost incident that motivated §8 has not recurred in the months since the policy shipped (an operational observation, not a controlled measurement). The naming pathology that motivated §5 — 259 accumulated single-use identities — is being retired as threads migrate to durable identities. We offer these less as results than as evidence that the requirements in §2.2 are the right ones to design against.

12 Conclusion

Agent frameworks have largely solved the inside of a task. Posse targets the outside: keeping many agents working, cheaply, for a long time, under a human who directs but does not supervise. Three decisions carry the design — identity decoupled from process liveness, token cost as a scheduled resource, and a zero-cost mechanical control plane separated from paid cognition. Together they turn four problems that sink a naive port of a session-based framework — thread continuity, many-thread management, continuous autonomy, and token efficiency — into ordinary, engineered system properties. The Sheriff watches; the deputies work; the records remember; and the invoice stays sane.

References

- [1] Significant Gravitas. *AutoGPT: An Autonomous GPT-4 Experiment*. 2023. <https://github.com/Significant-Gravitas/AutoGPT>.
- [2] Y. Nakajima. *BabyAGI*. 2023. <https://github.com/yoheinakajima/babyagi>.
- [3] S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2023.
- [4] N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS, 2023.

- [5] S. Yao et al. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS, 2023.
- [6] G. Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. arXiv:2305.16291, 2023.
- [7] J. S. Park et al. *Generative Agents: Interactive Simulacra of Human Behavior*. UIST, 2023.
- [8] Q. Wu et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv:2308.08155, 2023.
- [9] G. Li et al. *CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society*. NeurIPS, 2023.
- [10] S. Hong et al. *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. ICLR, 2024.
- [11] C. Qian et al. *Communicative Agents for Software Development (ChatDev)*. ACL, 2024.
- [12] J. Moura et al. *CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents*. 2023. <https://github.com/crewAIInc/crewAI>.
- [13] LangChain. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. 2024. <https://github.com/langchain-ai/langgraph>.
- [14] OpenAI. *Swarm: Educational Framework for Lightweight Multi-Agent Orchestration*. 2024. <https://github.com/openai/swarm>.
- [15] Anthropic. *Claude Code and the Claude Agent SDK*. 2024–2026.
- [16] K. Mei et al. *AIOS: LLM Agent Operating System*. arXiv:2403.16971, 2024.
- [17] C. Packer et al. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.
- [18] OpenAI. *Assistants API: Persistent Threads for Agents*. 2023–2024. <https://platform.openai.com/docs/assistants>.
- [19] Temporal Technologies. *Temporal: Durable Execution*. 2020–. <https://temporal.io>.
- [20] Amazon Web Services. *AWS Step Functions*. 2016.
- [21] M. Beauchemin et al. *Apache Airflow*. 2015. <https://airflow.apache.org>.
- [22] J. Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, KTH, 2003.
- [23] C. Hewitt, P. Bishop, R. Steiger. *A Universal Modular ACTOR Formalism for Artificial Intelligence*. IJCAI, 1973.
- [24] A. Verma et al. *Large-scale Cluster Management at Google with Borg*. EuroSys, 2015.
- [25] Anthropic. *Prompt Caching (cache-control, ~5-minute TTL)*. 2024. <https://docs.anthropic.com>.